

write a C program to simulate page replacement algorithms.

- i. FIFO
- ii. LRU
- iii. Optimal.

```
#include <stdio.h>
#define MAX 100
int pages[MAX], FRAMES, PAGES,
inFrames (int frames[], int page) {
    for (int i = 0; i < FRAMES; i++)
        if (frames[i] == page) return 1;
    return 0;
}

void FIFO() {
    int frames[FRAMES];
    for (int i = 0; i < FRAMES; i++) frames[i] = -1;

    int faults = 0, ptr = 0;
    printf("In -- FIFO -- \n");
    for (int i = 0; i < PAGES; i++) {
        if (!inFrames(frames, pages[i])) {
            frames[ptr] = pages[i];
            ptr = (ptr + 1) % FRAMES;
            faults++;
            printf("Page %d -> ", pages[i]);
            for (int j = 0; j < FRAMES; j++)
                printf("%d ", frames[j]);
            printf("FAULT \n");
        } else {
            printf("Page %d -> ", pages[i]);
            for (int j = 0; j < FRAMES; j++)
```



```

        printf("%d", frames[j]);
        printf("Hit\n");
    }
}

printf("Total faults: %d\n", faults);
}

void LRU() {
    int frames[FRAMES], lastUsed[FRAMES];
    for (int i = 0; i < FRAMES; i++) {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    int faults = 0;
    printf("\n--LRU--\n");
    for (int i = 0; i < PAGES; i++) {
        if (!inFrames(frames, pages[i])) {
            int replace = 0;
            for (int j = 0; j < FRAMES; j++) {
                if (frames[j] == -1) {
                    replace = j;
                    break;
                }
            }
            if (lastUsed[j] < lastUsed[replace])
                replace = j;

            frames[replace] = pages[i];
            lastUsed[replace] = i;
            faults++;
        }

        printf("Pages %d -> ", pages[i]);
    }
}

```



```

for(int j=0; j<FRAMES; j++)
    printf("%d", frames[j]);
printf("FAULT\n");
} else {
    for(int j=0; j<FRAMES; j++)
        if(frames[j]==pages[i])
            lastUsed[j]=i;
    printf("Pages %d -> ", pages[i]);
    for(int j=0; j<FRAMES; j++)
        printf("%d", frames[j]);
    printf("HIT\n");
}
}

```

```

printf("Total Faults: %d\n", faults);

```

```

void Optimal() {

```

```

    int frames[FRAMES];

```

```

    for(int i=0; i<FRAMES; i++) frames[i] = -1;

```

```

    int faults=0, filled=0;

```

```

    printf("\n--OPTIMAL--\n");

```

```

    for(int i=0; i<PAGES; i++) {

```

```

        if(!inFrames(frames, pages[i])) {

```

```

            if(filled < FRAMES) {

```

```

                frames[filled++] = pages[i];

```

```

            } else {

```

```

                int replace=0; farthest=-1;

```

```

                for(int j=0; j<FRAMES; j++) {

```

```

                    int nextUse = PAGES;

```

```

for (int k = i + 1; k < PAGES; k++)
{
    if (pages[k] != frames[j])
        nextUse = k;
        break;
}
}
if (nextUse > farthest) {
    farthest = nextUse;
    replace = j;
}
}
frames[replace] = pages[i];
}
faults++;
printf("Page %d -> ", pages[i]);
for (int j = 0; j < FRAMES; j++)
    printf("%d", frames[j]);
printf(" FAULT\n");
else {
    printf("Page %d -> ", pages[i]);
    for (int j = 0; j < FRAMES; j++)
        printf("%d", frames[j]);
    printf(" FA HIT\n");
}
}
printf("Total Faults: %d\n", faults);
}

int main() {
    printf("Enter number of frames: ");
    scanf("%d", &FRAMES);

```



```
printf("Enter number of pages:");
scanf("%d", &PAGES);
```

```
printf("Enter page reference string:\n");
for(int i=0; i<PAGES; i++){
    scanf("%d", &pages[i]);
}
```

```
FIFO();
```

```
LRU();
```

```
Optimal();
```

```
return 0;
```

```
}
```

output:

Enter number of pages: 20

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter number of frames: 3.

-- FIFO --

Page 7 → 7 -1 -1 FAULT

Page 0 → 7 0 -1 FAULT

Page 1 → 7 0 1 FAULT

Page 2 → 2 0 1 FAULT

Page 0 → 2 0 1 ~~MISS HIT~~

Page 3 → 2 3 1 FAULT

Page 0 → 2 3 0 FAULT

Page 4 → 4 3 0 FAULT

Page 2 → 4 2 0 FAULT

Page 3 → 4 2 3 FAULT

7	7	7	2
0	0	0	0
1	1	1	1

Page 0 → 0 2 3 FAULT

Page 3 → 0 2 3 HIT

Page 2 → 0 2 3 HIT

Page 1 → 0 1 3 FAULT

Page 2 → 0 1 2 FAULT

Page 0 → 0 1 2 HIT

Page 1 → 0 1 2 HIT

Page 7 → 7 1 2 FAULT

Page 0 → 7 0 2 FAULT

Page 1 → 7 0 1 FAULT

Total page faults: 15

--LRU--

Page 7 → 7 -1 -1 FAULT

Page 0 → 7 0 -1 FAULT

Page 1 → 7 0 1 FAULT

Page 2 → 2 0 1 FAULT

Page 0 → 2 0 1 HIT

Page 3 → 2 0 3 FAULT

Page 0 → 2 0 3 HIT

Page 4 → 4 0 3 FAULT

Page 2 → 4 0 3 FAULT

Page 3 → 4 3 2 FAULT

Page 0 → 0 3 2 FAULT

Page 3 → 0 3 2 HIT

Page 2 → 0 3 2 HIT

Page 1 → 1 3 2 FAULT

Page 2 → 1 3 2 HIT

Page 0 → 1 0 2 FAULT

Page 1 → 1 0 2 HIT

Page 7 → 7 0 2 FAULT

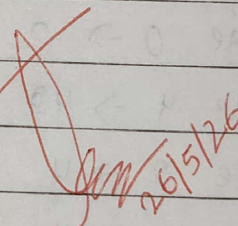
7	7	7
	0	0
	1	1

Page 0 $\rightarrow$ 1 0 7 HIT
 Page 1 $\rightarrow$ 1 0 7 HIT
 Total Page Faults = 12.

--OPTIMAL--

Page 7 $\rightarrow$ 7 -1 -1 FAULT
 Page 0 $\rightarrow$ 7 0 -1 FAULT
 Page 1 $\rightarrow$ 7 0 1 FAULT.
 Page 2 $\rightarrow$ 2 0 1 FAULT
 Page 0 $\rightarrow$ 2 0 1 HIT
 Page 3 $\rightarrow$ 2 0 3 ~~HIT~~ FAULT
 Page 0 $\rightarrow$ 2 0 3 HIT
 Page 4 $\rightarrow$ 2 4 3 FAULT
 Page 2 $\rightarrow$ 2 4 3 HIT
 Page 3 $\rightarrow$ 2 4 3 HIT
 Page 0 $\rightarrow$ 2 0 3 FAULT
 Page 3 $\rightarrow$ 2 0 3 HIT
 Page 2 $\rightarrow$ 2 0 3 HIT
 Page 1 $\rightarrow$ 2 0 1 FAULT
 Page 2 $\rightarrow$ 2 0 1 HIT
 Page 0 $\rightarrow$ 2 0 1 HIT
 Page 1 $\rightarrow$ 2 0 1 ~~HIT~~ FAULT
 Page 7 $\rightarrow$ 7 0 1 FAULT
 Page 0 $\rightarrow$ 7 0 1 HIT
 Page 1 $\rightarrow$ 7 0 1 HIT
 Total page faults = 9.

7	7	7
	0	0
		1


 26/5/26